FP 7 (Haskell 3): Working with lists & lambdas

Cael Marquard mrqcae001@myuct.ac.za
Department of Computer Science
University of Cape Town
(slides adapted from Dr Sabre Didi)



Announcement about reading & exercises

From now on, reading & exercises for each day will be put into a separate “Haskell exercises & readings” content item, at the end of the FP folder.

This is because apparently there is no easy way (that I’m aware of) to send an announcement to students only

Recap

1. (1 point) Suppose we have a value `x` of type `Int -> Int -> Int`. How would we express in Haskell that `x` has this type? Here, … is a placeholder for the value of `x` in an assignment
   - **a.** `let x: Int -> Int -> Int = ...`
   - **b.** `(Int -> Int -> Int) x = ...`
   - **c.** `x :: Int -> Int -> Int = ...`
   - **d.** `x {Int -> Int -> Int} = …`
2. (2 points) What is a sum type?
   - **a.** A generic type, with a filled-in type parameter, e.g. `List Int`
   - **b.** A type which must have a value for each and every type it is a sum of
   - **c.** A type that is able to be added with +, e.g. `Int`, `Float`, `Integer`, `Double`
   - **d.** A type which must have a value for one of the types it is a sum of

Aim of this lecture

At the end of this lecture you should:

- Know how to filter, map, fold, and zip with lists
- Know how to define lambdas (anonymous functions)
- Understand list, set, and string comprehensions
- Be able to use dependent generators and guards in said comprehensions

Map and filter

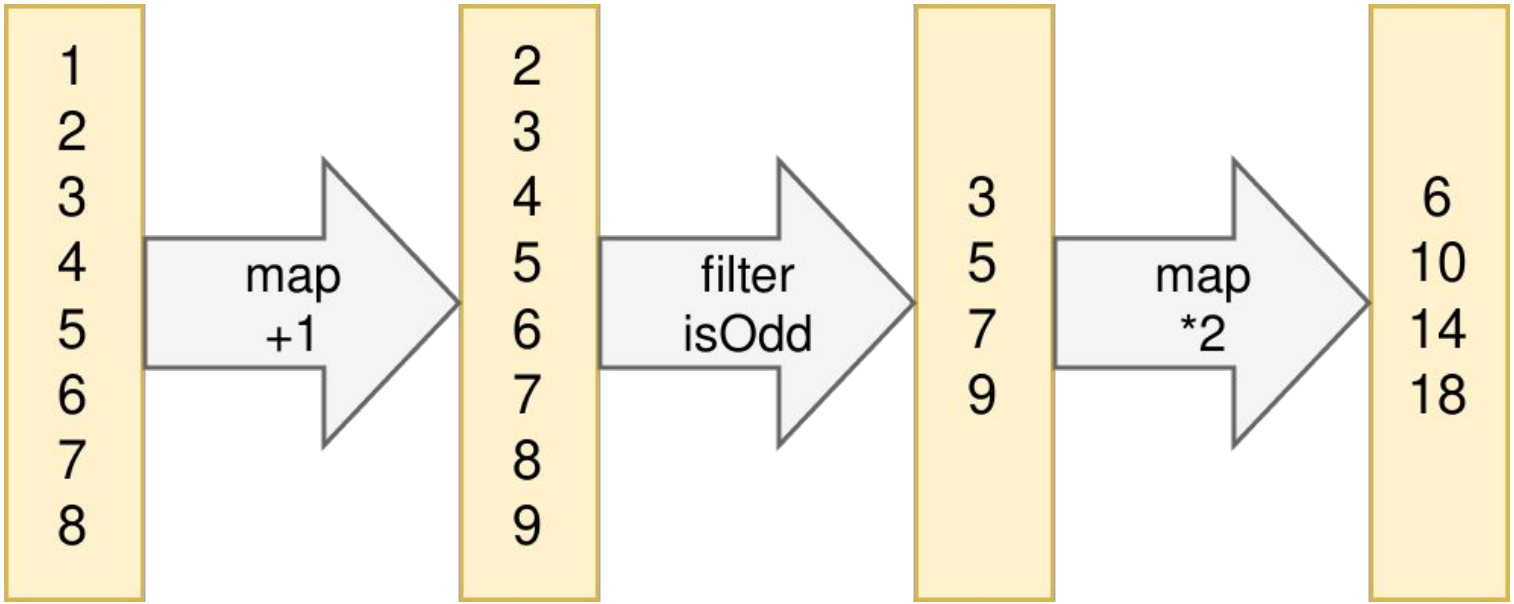
We're going to look at two **higher-order** functions in Haskell: map and filter. These work on sequences like lists and strings.

- **Higher-order function:** a function that accepts a function as an argument.
- **Map:** applies the given function (the **mapper**) to each element of a sequence
- **Filter:** applies the given function (the **predicate**) to each element of a sequence, and removes the element if the predicate returns False.
- Map, filter, and the functions like them are sometimes called (list) **combinators**

Therefore:

- `map (* 2) [1, 2, 3, 4]` would return `[2, 4, 6, 8]`
- `filter (<= 2) [1, 2, 3, 4]` would return `[1, 2]`

But why are these *practically* useful?



https://www.channable.com/tech/parallel-streaming-in-haskell-part-1-fast-efficient-fun

Batch processing with map and filter

Let's look at **batch processing** in Haskell. Suppose we need to **get the unique prime factor of a number** (so for 9, we would return just [3], not [3, 3])**.** A naive* approach to solving this is:

1. Get each factor of each number N
    a. Return all numbers in 1..=N that divide N
2. For each factor F, filter out non-prime factors
    a. Check that, out of 1..=F, only 1 and F divide F

We can translate this into thinking with filters & maps as follows

1. **map** our input list to **its factors**
    a. **Filter** numbers in 1..=N so they **divide N**
2. **filter** the factors so they are **prime**
    a. **Filter** numbers in 1..=F so they **divide F** and check this is equal to [1, F]
    b. Check that this resulting list **is empty** (= F is prime)

*obviously this can be done quicker with some optimisations, but that would needlessly complicate the example here

Prime factorisation in Haskell

1. **map** our input list to **its factors**
   a. **Filter** numbers in 1..=N so they **divide N**
2. **filter** the factors so they are **prime**
   a. **Filter** numbers in 1..=N so they **divide F** and check this is equal to [1, N]
   b. Check that this resulting list **is empty** (= F is prime)

Let's look at Q1 and Q2 as a class.

```haskell
primeFactoriseNums :: [Int] -> [[Int]]
primeFactoriseNums ns = map primeFactorise ns

primeFactorise :: Int -> [Int]
primeFactorise n = filter isPrime (factorise n)

factorise :: Int -> [Int]
-- Q1. What does "divides n" do? Why?
factorise n = filter (divides n) [1..n]

divides :: Int -> Int -> Bool
-- Q2. What do the ``s do here? Why?
divides num denom = num `mod` denom == 0

isPrime :: Int -> Bool
isPrime n = (factorise n) == [1, n]
```

Prime factorisation: summarising

This solution works, but we can make it shorter. Notice how we have this function which is only being defined in order to have it as an argument to `filter`?

```haskell
primeFactoriseNums :: [Int] -> [[Int]]
primeFactoriseNums ns = map primeFactorise ns

primeFactorise :: Int -> [Int]
primeFactorise n = filter isPrime (factorise n)

factorise :: Int -> [Int]
factorise n = filter (divides n) [1..n]

divides :: Int -> Int -> Bool
divides num denom = num `mod` denom == 0

isPrime :: Int -> Bool
isPrime n = (factorise n) == [1, n]
```

Lambdas

We can use a *lambda* here to replace our explicit `divides` function.

We call these functions “lambdas” after the greek letter of the same name (**λ**). Haskell is based on a theory of programming called **lambda calculus.**

A LAMBDA IS AN ANONYMOUS FUNCTION. THAT IS ALL!

```haskell
primeFactoriseNums :: [Int] -> [[Int]]
primeFactoriseNums ns = map primeFactorise ns

primeFactorise :: Int -> [Int]
primeFactorise n = filter isPrime (factorise n)

factorise :: Int -> [Int]
factorise n = filter (\f -> n `mod` f == 0) [1..n]

isPrime :: Int -> Bool
isPrime n = (factorise n) == [1, n]
```

Notations and names for lambdas

Lambdas are anonymous functions.

You might see them written as:

- `lambda x: 2 * x` in Python
- `x -> 2 * x` in Java
- `(x) => 2 * x` in Javascript (called closures)
- `|x| 2 * x` in Rust (called closures or the Fn* family)
- x ↦ 2x in Mathematics
- **or `\x -> 2 * x` in Haskell, of course!**

In Lambda calculus, two common notations are (λx.2x) and λx⟶2x

Why have lambdas?

1. We can reduce the number of one-off functions we need to define
2. We can use it to notate what **function currying** does:

   `add x y = x + y`

   … is equivalent to …

   add = λx → (λy → x + y)

(see how it was so easy to notate “a function returning a function” using a lambda?)

```haskell
primeFactoriseNums :: [Int] -> [[Int]]
primeFactoriseNums ns = map primeFactorise ns

primeFactorise :: Int -> [Int]
primeFactorise n = filter isPrime (factorise n)

factorise :: Int -> [Int]
factorise n = filter (\f -> n `mod` f == 0) [1..n]

isPrime :: Int -> Bool
isPrime n = (factorise n) == [1, n]
```

Why have lambdas? (2)

3. We can use it to write down functions returning other functions. For example consider a UI framework which lets us define the callback for a button. It gives us the number of times it's been pressed (Int), and our callback returns something to put on the screen (String)

```
makeCallback :: String -> (Int -> String)
makeCallback name = \n -> "Hello" ++ name ++
                          "! Button has been pressed" ++ show n ++ "times."

makeButton (makeCallback "Cael")
```

This can sometime show our intent a bit better than having makeCallback a two-arg function and then currying it. So, not a question of *what can be done,* but ergonomics and **expressivity** (something highly valued in FP for obvious reasons)**.**

Another list combinator: zip

Sometimes you need to combine two parallel lists into one for further processing. For example, let's say we have a list of people's names, and a list of their ages. Then, we want to print out the **names** of all people under the **age** of 65.

```
names = ["John", "Eliza", "Muhammad", "Velesazi"]
ages  = [ 18,     102,        34,         68    ]
```

We can't just `filter` on `names`, because our predicate doesn't receive the age. We can't just filter on `ages`, because we don't have access to the name.

Zip allows us to combine the two lists pairwise:

```
zip names ages -- returns [("John", 18), ("Eliza", 102),
               --         ("Muhammad", 34), ("Velezazi", 68)]
```

Zip (2)

```
zip names ages -- returns [("John", 18), ("Eliza", 102),
               --         ("Muhammad", 34), ("Velezazi", 68)]
```

Now, we can filter + map our zipped list.

```
map fst $ filter (\(name, age) -> age < 65) $ zip names ages
-- returns ["John", "Muhammad"]
```

Recall:

- `fst` returns the first element of a pair (2-tuple)
- `foo $ bar baz quux` is syntax sugar for `foo (bar baz quux)`

Quickfire combinators

Here are some other useful combinators to know:

- `all pred list` - returns true if `pred` is true for **all** elts of `list`
- `any pred list` - returns true if `pred` is true for **any** elts of `list`
- `take n list` - returns first n elts of `list`
- `drop n list` - removes first n elts of `list`
- `takeWhile pred list` - returns elts from `list` that match `pred`. Stops at first non-matching element.
- `dropWhile pred list` - remove elts from `list` that match `pred`. Stops at first matching element.

Fold: the general solution

Sometimes you need to do something that doesn't cleanly fit into zip, filter, map, or any of the other combinators (there are more that I haven't mentioned here).

For example, let's say we want to greet every person by name in a list, and output our final greeting paragraph.

```
names = ["Tshepo", "Tafadzwa", "Tom"]
-- we want "Hello Tshepo, Tafadzwa, Tom, and everyone else!"
```

This is a time for fold - the most powerful of all list combinators!

Fold (2)

```
names = ["Tshepo", "Tafadzwa", "Tom"]
(foldl (\acc elt -> acc ++ elt ++ ", ") "Hello, " names)
    ++ "and everyone else!"
-- outputs "Hello Tshepo, Tafadzwa, Tom, and everyone else!"
```

Fold takes three arguments:

1. Our operation (let's call it **f**)
2. The **accumulator** (often called **acc**)
3. Our **data** to fold over

For each elt of the data, it runs **`f acc elt`**, and uses the output of *that* as the next **acc.** You can think of it as being defined by the recurrence **acc' = f(acc, elt)**

Fold to implement product

Let’s say we want to take the product of a bunch of numbers. We can do this with fold easily:

```
ns = [5, 10, 2, 3]

foldl (*) 1 ns

-- returns 300
```

… but **what does this decompose to?**

Expanding fold: left vs right

*We will cover foldl' later in our lazy eval lecture!

In our product example:

```
ns = [5, 10, 2, 3]

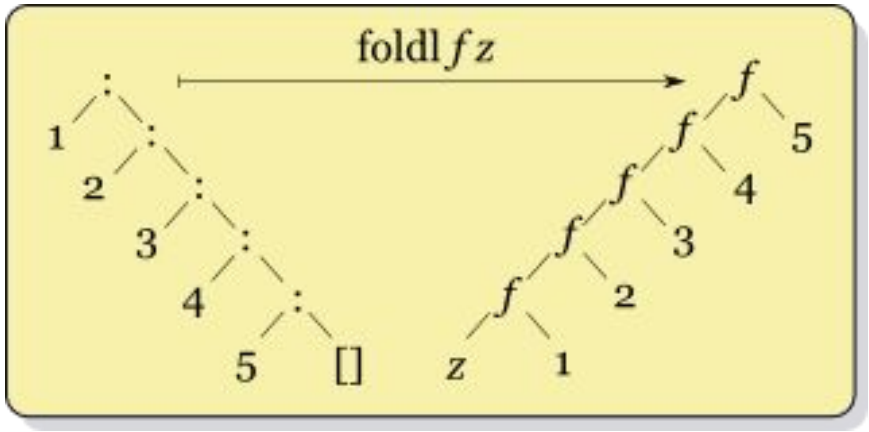
foldl (*) 1 ns
```

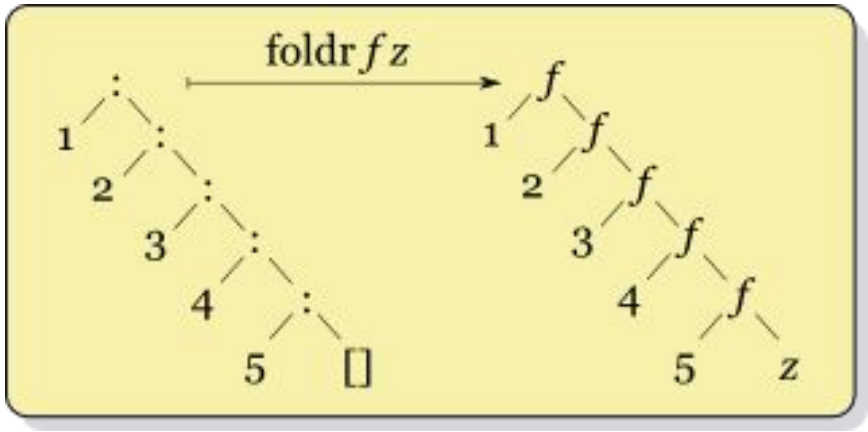
We obviously know that we are computing 1 * 5 * 10 * 2 * 3 = 300. But in which **order** does this get completed?

You may have noticed that our function is called **`foldl`**. That "L" stands for "Left", because it is **left-associative**. I.e., we are really computing (((1 * 5) * 10) * 2) * 3.

We can also do a **right-associative** fold with **`foldr`**: 5 * (10 * (2 * (3 * 1))). The

The difference matters for **non-associative/commutative** operators like subtraction. Let's stop and think about why **`foldl (-) 1 ns`** returns -19, but **`foldr (-) 1 ns`** returns -5





https://wiki.haskell.org/index.php?title=Fold

Fold is the universal list combinator

Fold is the universal list combinator. **Fold can implement *any other list combinator*** because of how general it is. **It is our functional-programming analog to a for-loop with state**, like this python code:

```
greeting = "Hello, "
for name in names:
    greeting += name + ","
greeting += "and everyone else"
```

The difference is just that **instead of mutating our accumulator** (`greeting`, here), **we are creating a copy that gets passed forward for each element.**

Sequence comprehensions

We've discussed how to use **combinators** to work with sequences (lists, strings, etc), but Haskell has another tool for us: **comprehensions.**

This is similar to how sets can be defined in mathematics. Consider this definition of the even, positive integers (E):

$$E = \{ 2x \mid x \in \mathbf{N} \}$$

What this means is that the set of even integers, E, can be obtained by taking every element of the positive integers ($x \in \mathbf{N}$), and doubling it (2x).

We express this in Haskell as follows:

```
e = [2 * x | x <- [1..]]
take 3 e -- prints [2, 4, 6]
```

This is a called a **"generator"**

Sequence comprehensions vs combinators

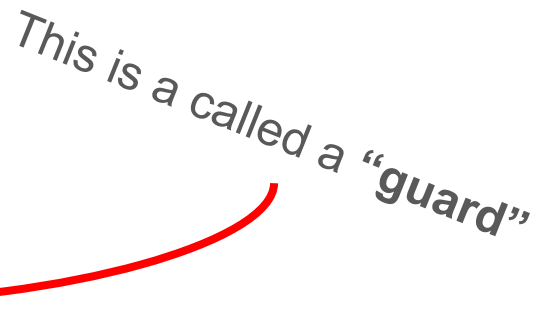
Sequence comprehensions are **an alternative to using combinators like map and filter.**

For example, we can return to our factorise function from earlier, and reimplement it with a comprehension:

```
-- type signatures are optional, even for functions,
-- if they can be inferred!

factorise n = filter (\f -> n `mod` f == 0) [1..n]

factorise' n = [f | f <- [1..n], n `mod` f == 0]
```



Let’s stop and think about how we could map a list of strings to their lengths.

Multiple inputs (NOT THE SAME AS ZIP!)

We can also give generators multiple inputs, allowing us to calculate the **cartesian product** of two inputs (every possible pair/tuple combining the input sets). This works like a **nested for-loop** (so, similarly, **order matters!**)

```
suits = ["Clubs", "Hearts", "Spades", "Diamonds"]
-- deck without jokers, aces, or royalty
regularCards = [(n, suit) | suit <- suits, n <- [2..10]]

take 3 regularCards
- prints [("2", "Clubs"), ("3", "Clubs"), ("4", "Clubs")]

take 3 $ drop 9 regularCards
-- prints [("2", "Hearts"), ("3", "Hearts"), ("4", "Hearts")]
```

Let's stop and think about what would happen if we reordered the inputs

Dependent generators

The input generators can also **depend on other input generators!** Provided they come after in the inputs, of course.

Let's say we wanted to list which **coordinates (row, column)** in a 3x3 **triangular matrix** (bottom/left aligned) are **equal to 1**.

$$\begin{Bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{Bmatrix}$$

```
[(x, y) | x <- [1..3], y <- [1..x]]
-- prints [(1, 1),
--         (2, 1), (2, 2),
--         (3, 1), (3, 2), (3,3)]
```

Let's stop and think about how we could flatten a list of lists using dependent generators (e.g. flatten [[1, 2], [3, 4], [5]] should return [1, 2, 3, 4, 5])

String comprehensions

A string is just a list of characters. We can use map, filter, and fold on it. We can also use comprehension to build a string:

```
import Data.Char
[toUpper c | c <- "hello"]
-- prints "HELLO"
```

Exercise

(please remember that if you use AI to do the following, it no longer has any value. You are only cheating yourself.)

Prove that every list combinator is really just a fold by reimplementing the `sum`, `map`, and `filter` functions in terms of `foldl`. Recall that `list ++ [item]` or `item:list` can be used to add items to a list.

Assigned reading & assistance

As revision for today:

- (LYAH) Starting Out: I'm a list comprehension
- (LYAH) Starting Out: An intro to lists

As preparation for next lecture:

- (LYAH) Types and Typeclasses (entire chapter)
- (LYAH) Syntax in Functions (entire chapter)

If you have any queries, email them to me at mrqcae001@myuct.ac.za. I am happy to do office hours by appointment (also by email)